

Лабораторная работа №7

Дискретное логарифмирование в конечном поле

11 Марта 2026 - Ли Хан

2026

Российский университет дружбы народов, Москва, Россия

Цель работы

В рамках данного эксперимента я исследовал и реализовал задачу дискретного логарифмирования в конечном поле F_p . Суть задачи состоит в нахождении такого целого числа x , что $a^x \equiv b \pmod{p}$. Безопасность таких протоколов, как Диффи-Хеллман, основана на том, что возведение в степень выполняется легко, а решение задачи дискретного логарифмирования — крайне сложная вычислительная задача.

Детальный разбор алгоритмов

Наиболее важным шагом в реализации алгоритма является построение псевдослучайного отображения $f(c)$, обладающего свойством «вычислимости логарифма».

- Логика кода: Я реализовал ветвящуюся функцию. В зависимости от положения текущего значения c в поле (например, относительно $p/2$), выполняется либо умножение на a , либо на b .
- Глубокий разбор: Если мы представим c как $a^u b^v \pmod{p}$, то на каждом шаге итерации мы можем точно отслеживать изменения показателей u и v . Например, умножение на a эквивалентно увеличению u на 1, а умножение на b — увеличению v на 1. Таким образом, мы превращаем простую смену значений в процесс отслеживания линейной функции относительно неизвестного x .

Чтобы найти дискретный логарифм, нам нужно найти две разные комбинации показателей, которые дают один и тот же элемент группы.

- Логика кода: Я использовал алгоритм Флойда. Медленный указатель c делает один шаг f за итерацию, а быстрый d — два шага.
- Глубокий разбор: Когда $c \equiv d \pmod{p}$, мы получаем коллизию. Это дает нам равенство $a^{u_c}b^{v_c} \equiv a^{u_d}b^{v_d} \pmod{p}$. Этот момент является ключевым, так как он переводит задачу поиска логарифма в плоскость решения линейного сравнения.

Как только коллизия обнаружена, я извлекаю уравнение для показателей степеней.

- Логика кода: Я составляю уравнение $(v_d - v_c)x \equiv (u_c - u_d) \pmod{r}$, где r — порядок числа a .
- Глубокий разбор: Для решения этого линейного сравнения я вызываю расширенный алгоритм Евклида. Этот шаг является выходом из алгоритма: путем вычисления обратного элемента коэффициента мы получаем значение x .

Итоги работы

При тестировании примера из пособия $10^x \equiv 64 \pmod{107}$ мой код обнаружил коллизию на 11-м шаге. Решив сравнение $7 + 8x \equiv 13 + 13x \pmod{53}$, я получил $x = 20$. Проверка $10^{20} \pmod{107}$ подтвердила результат 64, что доказывает эффективность ρ -метода Полларда для решения подобных задач.